Numerical Methods for Ordinary Differential Equations

Class Notes Reference

Quick Reference: Euler Methods

For an initial value problem where time is discretized in steps of Δt ($t_{n+1} = t_n + \Delta t$), the fundamental first-order update schemes are:

Explicit Euler Method: Evaluates the derivative at the current known state.

$$y_{n+1} = y_n + \Delta t\, y'(t_n, y_n) \tag{1}$$

Implicit (Backward) Euler Method: Evaluates the derivative at the future, unknown state (requires solving an equation).

$$y_{n+1} = y_n + \Delta t\, y'(t_{n+1}, y_{n+1}) \tag{2}$$

Semi-Implicit (Symplectic) Euler Method: Evaluates momentum/velocity explicitly, then uses the updated velocity to find the new position. Highly desirable for conservative Hamiltonian systems.

$$v_{n+1} = v_n + a(t_n, y_n)\Delta t \tag{3}$$
$$y_{n+1} = y_n + v_{n+1}\Delta t \tag{4}$$

---

1 Euler's Method and Numerical Uncertainty

When solving physical systems, higher-order ordinary differential equations (ODEs) must first be reduced to a system of first-order equations. The simplest approach to numerical integration is the **Explicit Euler Method**.

For an initial value problem, time is discretized in steps of Δt such that $t_{n+1} = t_n + \Delta t$. The explicit update rule evaluates the derivative strictly at the current, known position:

$$y_{n+1} = y_n + \Delta t\, y'(t_n, y_n) \tag{5}$$

Because the right-hand side depends only on the current state (the past), it is termed an *explicit* method.

2 Error Analysis: The Falling Body

Consider a falling object (e.g., a falling cow) with no air drag. We separate the motion into x and y components. We can derive the exact analytical solution by expanding the position via a Taylor series around t_0:

$$y(t_0 + \Delta t) = y(t_0) + y'(t_0)\Delta t + \frac{1}{2}y''(t_0)\Delta t^2 + \mathcal{O}(\Delta t^3) \tag{6}$$

Assuming uniform gravity, there are no higher-order terms past the second derivative, and the acceleration is constant, $y''(t_0) = -g$. Substituting this in yields the exact kinematic equation for one time step:

$$y_{\text{exact}} = y_n + y'_n\Delta t - \frac{1}{2}g\Delta t^2 \tag{7}$$

By contrast, the Explicit Euler method truncates the series after the first derivative, yielding:

$$y_{\text{explicit}} = y_n + y'_n\Delta t \tag{8}$$

2.1 Local vs. Global Truncation Error

The difference between the exact analytical step and the explicit Euler approximation yields the **Local Truncation Error** per step. This is the mathematical error introduced purely by truncating the infinite Taylor series:

$$E_{\text{local}} = |y_{\text{exact}} - y_{\text{explicit}}| = \left|-\frac{1}{2}g\Delta t^2\right| = \mathcal{O}(\Delta t^2) \tag{9}$$

To find the **Global Truncation Error** accumulated after a total integration duration T, we must sum the errors over N steps. The number of steps is $N = \frac{T}{\Delta t}$:

$$E_{\text{global}} \propto \sum_{i=1}^{N} \Delta t^2 = N\Delta t^2 = \left(\frac{T}{\Delta t}\right)\Delta t^2 = T\Delta t \tag{10}$$

Because T is a constant total time, the global truncation error scales linearly with the step size, making Explicit Euler a first-order method: $E_{\text{global}} = \mathcal{O}(\Delta t)$.

Note: The error analyzed here is strictly the mathematical truncation error. While this implies that mathematically making Δt infinitely small yields a perfect result, performing these algebraic steps exactly on a computer will introduce additional sources of error, which we will discuss next class.

3 Implicit Methods and Finite Differences

Implicit Methods evaluate the derivative at the *future* time step, requiring us to solve for the next state concurrently. The general form is:

$$y_{n+1} = y_n + \Delta t\, y'(t_{n+1}, y_{n+1}) \tag{11}$$

Let's walk through this for the falling object. The continuous velocity evolves as $y'(t_{n+1}) = y'(t_n) - g\Delta t$. Substituting this future velocity into the implicit update rule gives:

$$y_{\text{implicit}} = y_n + \Delta t\left[y'(t_n) - g\Delta t\right] \tag{12}$$

$$= y_n + y'(t_n)\Delta t - g\Delta t^2 \tag{13}$$

Now, let's explicitly compare the displacement term for all three approaches:

$$\begin{aligned} \text{Exact:} \quad & y_{n+1} = y_n + y'_n \Delta t - \frac{1}{2} g \Delta t^2 \\ \text{Explicit:} \quad & y_{n+1} = y_n + y'_n \Delta t + 0 \\ \text{Implicit:} \quad & y_{n+1} = y_n + y'_n \Delta t - g \Delta t^2 \end{aligned}$$

Notice that compared to the exact $-\frac{1}{2} g \Delta t^2$ term, the explicit method is off by $+\frac{1}{2} g \Delta t^2$, and the implicit method is off by $-\frac{1}{2} g \Delta t^2$. They diverge from the true solution in exactly opposite directions! Averaging the explicit and implicit steps effectively cancels this $\mathcal{O}(\Delta t^2)$ truncation error, forming the basis of higher-order methods like the Trapezoidal rule.

3.1 Relation to Derivative Forms

These integration methods map directly to finite difference approximations of the first derivative:

- **Forward Difference (Explicit):** $y'(t) \approx \frac{y(t+\Delta t) - y(t)}{\Delta t}$
- **Backward Difference (Implicit):** $y'(t) \approx \frac{y(t) - y(t-\Delta t)}{\Delta t}$

4 Relation to Integration

The fundamental problem of calculating a definite integral can be thought of as solving a very simple ordinary differential equation. If we want to evaluate $\int_a^b f(t)\, dt$, this is mathematically equivalent to solving the ODE:

$$y'(t) = f(t) \tag{14}$$

where the derivative $f(t)$ depends only on time, not on the state y.

If we apply the ODE numerical methods we have discussed to this simple equation, the equivalency to standard numerical quadrature (calculating the area under a curve) becomes explicit.

Applying the **Explicit Euler Method** to this ODE yields:

$$y_{n+1} = y_n + \Delta t\, f(t_n) \tag{15}$$

Summing these discrete steps over an interval is exactly equivalent to computing a **Left Riemann Sum**. The explicit method replaces the continuous area with discrete rectangles whose heights are determined by the function value at the *left* edge of each Δt step.

Conversely, applying the **Implicit Euler Method** yields:

$$y_{n+1} = y_n + \Delta t\, f(t_{n+1}) \tag{16}$$

This is exactly equivalent to a **Right Riemann Sum**, where the area is approximated using rectangles whose heights are taken from the *right* edge of the interval.

5 Semi-Implicit Methods and Symplectic Integration

A **Semi-Implicit** (or Symplectic Euler) scheme mixes the update timings. It updates the momentum (or velocity) explicitly, but then uses that *newly calculated* velocity to update the position:

$$v_{n+1} = v_n + a(y_n)\Delta t \tag{17}$$

$$y_{n+1} = y_n + v_{n+1}\Delta t \tag{18}$$

This slight change in timing has profound implications for Hamiltonian mechanics. For a conservative physical system, the state is described in phase space by its position q and momentum p, governed by the analytical Hamiltonian $H(q,p) = T(p) + V(q)$. While standard explicit or implicit Euler methods artificially inject or drain energy from a system over time, the Symplectic Euler method is designed to be volume-preserving in phase space.

5.1 The Falling Rock (Unbounded Phase Space)

For a falling rock in a uniform gravitational field, the analytical Hamiltonian is:

$$H(y,p) = \frac{p^2}{2m} + mgy \tag{19}$$

Because $p = mv$, this is equivalent to the total mechanical energy $E = \frac{1}{2}mv^2 + mgy$.

Let's calculate the energy evaluated at step $n+1$ using the semi-implicit updates ($a = -g$):

$$\begin{aligned}
E_{n+1} &= \frac{1}{2}m(v_n - g\Delta t)^2 + mg\left[y_n + (v_n - g\Delta t)\Delta t\right] \\
&= \frac{1}{2}m(v_n^2 - 2v_n g\Delta t + g^2\Delta t^2) + mg(y_n + v_n\Delta t - g\Delta t^2) \\
&= \left(\frac{1}{2}mv_n^2 + mgy_n\right) - mv_n g\Delta t + \frac{1}{2}mg^2\Delta t^2 + mgv_n\Delta t - mg^2\Delta t^2 \\
E_{n+1} &= E_n - \frac{1}{2}mg^2\Delta t^2
\end{aligned}$$

The $-mv_n g\Delta t$ and $+mgv_n\Delta t$ cross-terms exactly cancel. However, we are left with a constant energy drift per step of $\Delta E = -\frac{1}{2}mg^2\Delta t^2$. Because the falling rock is an *unbounded* system (the velocity increases and position decreases indefinitely to negative infinity), its phase-space trajectory is an open parabola. The symplectic nature of the integrator cannot prevent the global error from drifting quadratically over time.

5.2 The Harmonic Oscillator (Bounded Phase Space)

The true power of symplectic integration is revealed in bounded, periodic systems like a harmonic oscillator ($F = -ky$). The analytical Hamiltonian is:

$$H(y,p) = \frac{p^2}{2m} + \frac{1}{2}ky^2 \tag{20}$$

The exact phase-space trajectory (y,p) for this system forms a closed ellipse.

When we integrate this using Symplectic Euler, the method does not exactly conserve the analytical Hamiltonian H. Instead, it exactly conserves a **shadow Hamiltonian** $\tilde{H}$, which is a slightly perturbed version of the true energy:

$$\tilde{H} = H + \mathcal{O}(\Delta t) \tag{21}$$

Because Symplectic Euler mathematically preserves the geometric area in phase space ($dp \wedge dq$ is strictly conserved), the trajectory must also form a closed loop. It cannot spiral inward (losing energy) or spiral outward (gaining energy) like standard Euler methods. Consequently, if these algebraic steps are performed exactly, the numerical energy error never drifts to infinity; it remains strictly bounded, oscillating around the true analytical energy forever.

Caveat: While performing these mathematical steps exactly guarantees this bounded behavior, computational implementations on a computer will have additional sources of error that prevent a perfect closed system. We will discuss these computational errors next class.

6 Higher-Order Methods and the Runge-Kutta Family

To build methods with lower truncation error, we can generalize numerical integration by looking at the exact Taylor expansion of the solution up to the second order:

$$y(t_{n+1}) = y_n + \Delta t\, y'_n + \frac{1}{2}\Delta t^2\, y''_n + \mathcal{O}(\Delta t^3) \tag{22}$$

However, we can approximate the second derivative using a finite difference of the first derivative. Suppose we evaluate this finite difference over the *full step* (Δt):

$$y''_n \approx \frac{y'_{n+1} - y'_n}{\Delta t} \tag{23}$$

Substituting this into the Taylor expansion yields:

$$\begin{aligned}
y_{n+1} &\approx y_n + \Delta t\, y'_n + \frac{1}{2}\Delta t^2 \left(\frac{y'_{n+1} - y'_n}{\Delta t}\right) \\
y_{n+1} &\approx y_n + \Delta t\, y'_n + \frac{\Delta t}{2}\left(y'_{n+1} - y'_n\right) \\
y_{n+1} &\approx y_n + \Delta t\, y'_n + \frac{\Delta t}{2} y'_{n+1} - \frac{\Delta t}{2} y'_n \\
y_{n+1} &\approx y_n + \frac{\Delta t}{2}\left(y'_n + y'_{n+1}\right)
\end{aligned}$$

This is exactly the **Trapezoidal Rule** for integration (often implemented as Heun's Method for ODEs), achieving second-order accuracy by averaging the derivative at the beginning and the end of the step.

This principle is the foundation of the **Runge-Kutta (RK) family of integrators**. Heun's Method is a specific, second-order case of Runge-Kutta. By using carefully chosen finite difference approximations and intermediate evaluation stages (often called $k_1, k_2, k_3, \dots$), this extensible framework systematically matches higher-order Taylor series coefficients without ever needing to calculate analytical derivatives directly, allowing for higher order integrators.